

Sección 4: Realizar evaluación, análisis de errores y ajuste (15-20%)

🔗 **Esta sección evalúa tu capacidad para medir el rendimiento de un agente de IA, diagnosticar sus fallos y ajustar su comportamiento para mejorar resultados futuros.** Cubre la definición de criterios de éxito (cuantitativos y cualitativos), el uso de herramientas automatizadas de evaluación, el análisis de causas raíz de errores y las técnicas de ajuste fino en instrucciones, memoria y herramientas. Con un peso del 15-20% del examen, esta sección es crítica porque conecta directamente con la capacidad de operar agentes de IA de forma profesional y sostenible en entornos de producción.

4.1 Definición de criterios de éxito y señales de evaluación para tareas del agente

Conceptos clave

¿Por qué definir criterios de éxito?

Un agente de IA no puede mejorar si no se puede medir su rendimiento. Definir criterios de éxito **antes** de ejecutar una tarea permite:

- **Objetividad:** Evaluar el resultado contra métricas concretas, no contra impresiones subjetivas.
- **Reproducibilidad:** Aplicar los mismos criterios en diferentes ejecuciones para detectar regresiones.
- **Alineación:** Asegurar que lo que el agente produce está alineado con lo que el desarrollador realmente necesita.
- **Iteración informada:** Saber exactamente qué ajustar cuando los resultados no cumplen las expectativas.

💡 **Principio fundamental:** "Si no puedes medirlo, no puedes mejorarlo." Esto aplica directamente a los agentes de IA: sin métricas claras, el ajuste se convierte en prueba y error sin dirección.

Resultados esperados y restricciones operativas

Al asignar una tarea a un agente, se deben especificar dos dimensiones:

Dimensión	Definición	Ejemplo
Resultado esperado	Qué debe producir el agente al finalizar la tarea	"Un PR que implemente autenticación JWT con tests unitarios"
Restricción operativa	Límites dentro de los cuales el agente debe operar	"No modificar archivos fuera del directorio <code>src/auth/</code> ", "No instalar dependencias nuevas sin justificación"

Resultado esperado: Es el "qué" de la tarea. Debe ser específico, verificable y alineado con un issue o requisito documentado. Un resultado esperado mal definido genera ambigüedad, y el agente llenará esa ambigüedad con sus propias suposiciones.

Ejemplos de resultados esperados bien definidos:

- ✓ "Crear un endpoint GET /api/users que devuelva una lista paginada de usuarios activos."
- ✓ "Corregir el bug #142 donde el formulario de registro no valida emails duplicados."
- ✓ "Refactorizar la clase PaymentProcessor para usar el patrón Strategy."
- ✗ "Mejorar el código de autenticación." (demasiado vago)
- ✗ "Hacer que todo funcione mejor." (no verificable)

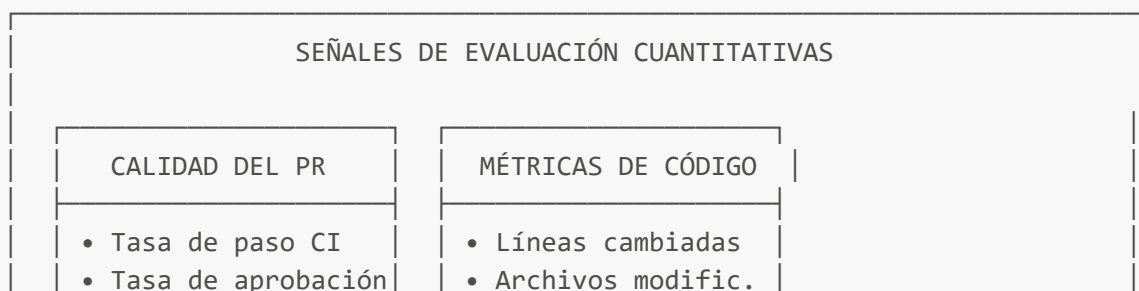
Restricción operativa: Es el "cómo no" de la tarea. Define los límites que el agente no debe cruzar. Las restricciones se implementan en GitHub Copilot a través de:

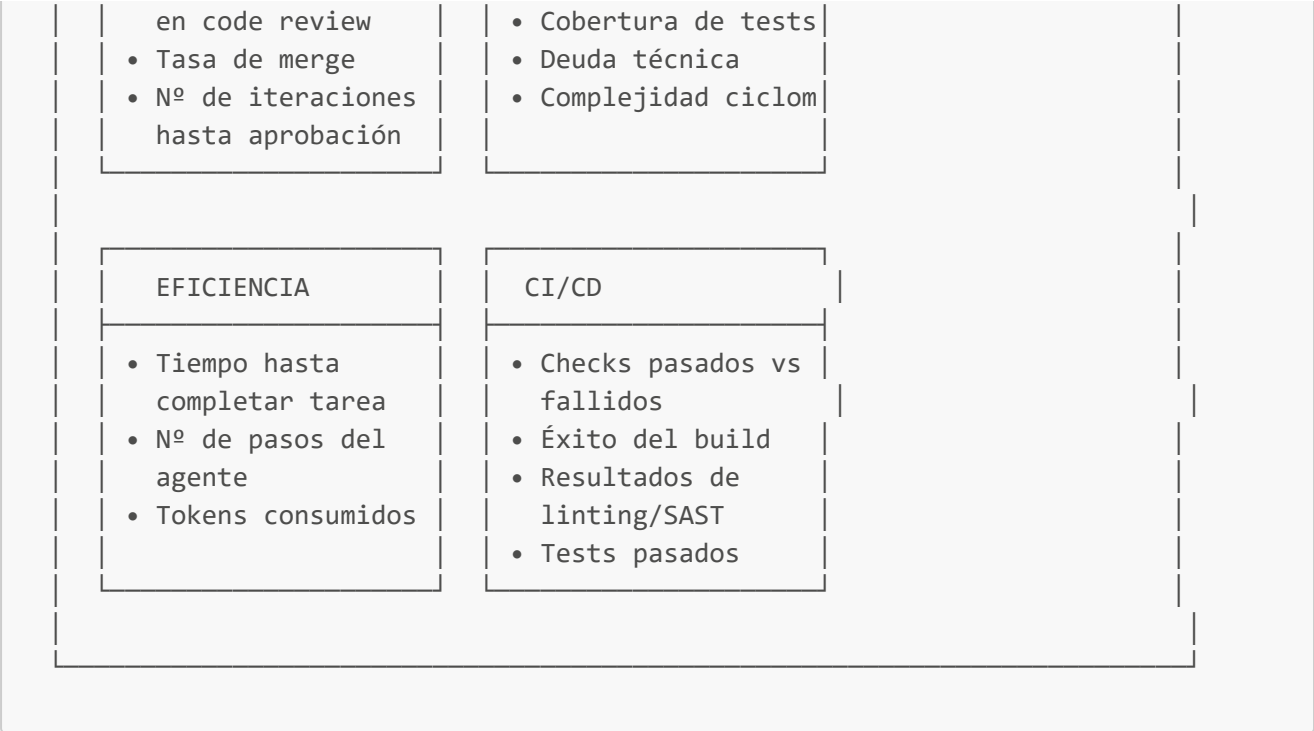
- **copilot-instructions.md:** Restricciones globales para todo el repositorio.
- **Archivos .github/instructions/*.instructions.md:** Restricciones específicas por tipo de archivo o directorio (usando patrones `applyTo` en el frontmatter).
- **Definiciones de agentes personalizados (.github/agents/*.agent.md):** Restricciones específicas por agente, incluyendo herramientas permitidas y prohibidas.
- **Descripción del issue:** Contexto y límites específicos para una tarea individual.

```
# Ejemplo en copilot-instructions.md
## Restricciones operativas generales
- No modificar archivos de configuración de infraestructura sin aprobación explícita.
- Toda función pública debe tener al menos un test unitario.
- No introducir dependencias con licencias incompatibles con MIT.
- Máximo 300 líneas por archivo nuevo creado.
```

Señales de evaluación cuantitativas

Las señales cuantitativas son métricas numéricas objetivas que se pueden medir automáticamente. En el ecosistema de GitHub, las principales son:





Detalle de métricas clave:

Métrica	Qué mide	Cómo se obtiene en GitHub	Valor ideal
Tasa de paso CI	% de ejecuciones de CI que pasan	GitHub Actions check runs en el PR	>95%
Tasa de aprobación en review	% de PRs aprobados sin cambios solicitados	Reviews del PR (APPROVED vs CHANGES_REQUESTED)	>80% en primera iteración
Tasa de merge	% de PRs del agente que se fusionan al main	Estado final del PR (merged vs closed)	>90%
Cobertura de tests	% del código nuevo cubierto por tests	Herramientas de cobertura integradas en CI	>80%
Líneas cambiadas	Volumen del cambio generado	PR diff stats (additions + deletions)	Proporcional a la tarea
Archivos modificados	Alcance del cambio	PR file list	Mínimos necesarios
Tiempo hasta completar	Eficiencia del agente	Timestamps del issue y del PR	Menor es mejor

Señales de evaluación cualitativas

Las señales cualitativas requieren juicio humano y no se pueden reducir a un número simple. Son igualmente importantes porque capturan aspectos que las métricas automáticas no detectan:

Señal cualitativa	Qué evalúa	Cómo se observa
Alineación arquitectónica	¿El código sigue los patrones y convenciones del proyecto?	Revisión de code review, comparación con código existente
Legibilidad	¿El código es fácil de entender y mantener?	Comentarios en code review, facilidad de onboarding
Nombramiento	¿Variables, funciones y clases tienen nombres descriptivos y consistentes?	Inspección manual del diff
Manejo de edge cases	¿El agente consideró casos límite y errores?	Tests generados, manejo de errores en el código
Documentación	¿Se actualizó la documentación relevante?	Archivos de docs en el PR, README actualizado
Alcance del cambio	¿El agente se mantuvo dentro del alcance solicitado?	Comparación del diff con el issue original
Comentarios de review	Feedback textual de los revisores humanos	Threads de review en el PR

💡 **Para el examen:** Diferencia claramente entre señales cuantitativas (medibles automáticamente, numéricas) y cualitativas (requieren juicio humano, descriptivas). El examen puede presentar escenarios donde debes identificar qué tipo de señal es más apropiada.

Alineación de criterios de evaluación con la intención de desarrollo

La **intención de desarrollo** (developer intent) es lo que el desarrollador realmente quiere lograr, no solo lo que dice literalmente. Los criterios de evaluación deben capturar esta intención, no solo la tarea superficial.

EJEMPLO DE DESALINEACIÓN:

Tarea: "Añadir validación al formulario de login"

Intención: Prevenir ataques de inyección y mejorar la UX mostrando mensajes de error claros.

Agente sin alineación:

- └ ☒ Añade validación de campos vacíos
- └ ☒ No sanitiza inputs contra XSS
- └ ☒ Mensajes de error genéricos ("Error en el campo")
- └ Resultado: Técnicamente cumple la tarea, pero no la intención.

Agente con alineación:

- └ ☒ Valida campos vacíos y formato de email
- └ ☒ Sanitiza inputs contra XSS e inyección SQL
- └ ☒ Mensajes de error descriptivos por campo

- └─ ☒ Tests para cada caso de validación
- └─ Resultado: Cumple la tarea Y la intención.

Cómo alinear los criterios:

1. **Definir la intención en el issue:** No solo describir "qué" hacer, sino "por qué" y "para quién". El agente utiliza esta información para tomar mejores decisiones.
2. **Incluir criterios de aceptación en el issue:** Lista de condiciones que deben cumplirse para considerar la tarea completa.
3. **Usar instrucciones personalizadas para codificar la intención:** Las instrucciones en `copilot-instructions.md` actúan como una capa permanente de intención del equipo.
4. **Evaluar contra la intención, no solo contra la tarea literal:** En la revisión del PR, preguntarse: "¿Esto resuelve el problema real?"

Generación de señales de evaluación mediante herramientas automatizadas

GitHub proporciona herramientas que generan señales de evaluación automáticamente, sin intervención humana:

1. GitHub Actions como puertas de calidad automatizadas (Quality Gates)

Los workflows de GitHub Actions ejecutan comprobaciones automáticas en cada push o PR. Cada check run genera una señal de evaluación:

```
# Ejemplo: Workflow de CI que genera señales de evaluación
name: Agent Output Evaluation
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      # Señal: ¿Compila correctamente?
      - name: Build
        run: npm run build

      # Señal: ¿Pasan los tests existentes?
      - name: Unit Tests
        run: npm test

      # Señal: ¿Cumple las reglas de estilo?
      - name: Lint
        run: npm run lint

      # Señal: ¿No introduce vulnerabilidades?
      - name: Security Scan
        run: npm audit --audit-level=high
```

```
# Señal: ¿Mantiene la cobertura de tests?  
- name: Coverage  
  run: npm run test:coverage
```

Cada paso que pasa o falla es una señal cuantitativa automática. El resultado agregado de todos los checks es visible en el PR como un estado combinado (success, failure, pending).

2. Copilot Code Review como evaluador automatizado

GitHub Copilot Code Review actúa como un revisor automatizado que genera señales cualitativas:

- **Se activa automáticamente** en PRs o se solicita manualmente asignando **copilot** como reviewer.
- **Analiza el diff** del PR y genera comentarios de revisión con sugerencias.
- **Detecta problemas** de rendimiento, seguridad, legibilidad y bugs potenciales.
- **Proporciona sugerencias de código** que el autor puede aceptar o rechazar.

Las señales que genera Copilot Code Review incluyen:

- Número de comentarios de revisión generados (más comentarios sugiere más problemas).
- Severidad de los problemas detectados.
- Tipos de problemas (seguridad, rendimiento, estilo, lógica).
- Sugerencias aceptadas vs. rechazadas (mide la utilidad del reviewer).

3. Branch Protection Rules y Required Status Checks

Las reglas de protección de ramas combinan múltiples señales en una decisión binaria: ¿se puede fusionar el PR o no?

Branch Protection para "main":

- └─ ☒ Requiere que CI pase (status checks obligatorios)
- └─ ☒ Requiere al menos 1 aprobación de code review
- └─ ☒ Requiere que Copilot Code Review no tenga errores críticos
- └─ ☒ Requiere que no haya conversaciones sin resolver
- └─ Resultado: Si todo pasa → merge permitido → señal de éxito

 **Concepto para el examen:** Las herramientas automatizadas de evaluación en GitHub forman un **pipeline de evaluación** que va desde señales individuales (cada check de CI) hasta una señal agregada (estado del PR). Este pipeline es análogo a un sistema de evaluación continua.

Aplicación práctica en GitHub

Escenario: Evaluación de un PR generado por Copilot coding agent

Un equipo asigna el issue #87 ("Implementar endpoint de paginación para /api/products") al agente de codificación de Copilot. El agente genera un PR. ¿Cómo evaluar el resultado?

Paso 1: Definir criterios antes de asignar la tarea

Issue #87: Implementar endpoint de paginación para /api/products

Descripción

El endpoint actual devuelve todos los productos. Necesitamos paginación para manejar catálogos grandes (>10,000 productos).

Criterios de aceptación

- [] Endpoint acepta parámetros `page` y `pageSize`
- [] Valores por defecto: page=1, pageSize=20
- [] Respuesta incluye metadata: totalItems, totalPages, currentPage
- [] Validación de parámetros (page >= 1, 1 <= pageSize <= 100)
- [] Tests unitarios para todos los casos
- [] No romper el comportamiento existente del endpoint
- [] Documentación de la API actualizada

Restricciones

- Usar el patrón de paginación offset-based existente en otros endpoints
- No modificar el esquema de la base de datos
- Máximo 5 archivos modificados

Paso 2: Verificar señales cuantitativas del PR

Señal	Esperado	Resultado del agente	¿Cumple?
CI checks	Todos pasan	8/8 checks pasan	☑
Tests nuevos	≥5 tests	7 tests añadidos	☑
Archivos modificados	≤5	4 archivos	☑
Cobertura de tests	>80%	92%	☑
Líneas añadidas	Proporcional	145 líneas	☑
Copilot Review comments	0 críticos	1 sugerencia menor	☑

Paso 3: Verificar señales cualitativas del PR

- ¿Sigue el patrón de paginación existente? → Sí, usa el mismo helper `paginate()`.
- ¿Mensajes de error claros? → Sí, devuelve 400 con descripción del error.
- ¿Documentación actualizada? → Sí, actualiza el archivo OpenAPI spec.
- ¿Nombres de variables consistentes? → Sí, sigue convenciones del proyecto.

Puntos clave para el examen

- **Definir criterios de éxito ANTES de ejecutar la tarea.** Los criterios retroactivos son menos útiles porque están sesgados por el resultado.
- Las señales cuantitativas se miden automáticamente; las cualitativas requieren juicio humano.
- Los criterios de evaluación deben reflejar la **intención de desarrollo**, no solo la tarea literal.
- **GitHub Actions** genera señales cuantitativas automáticas (build, tests, lint, seguridad).

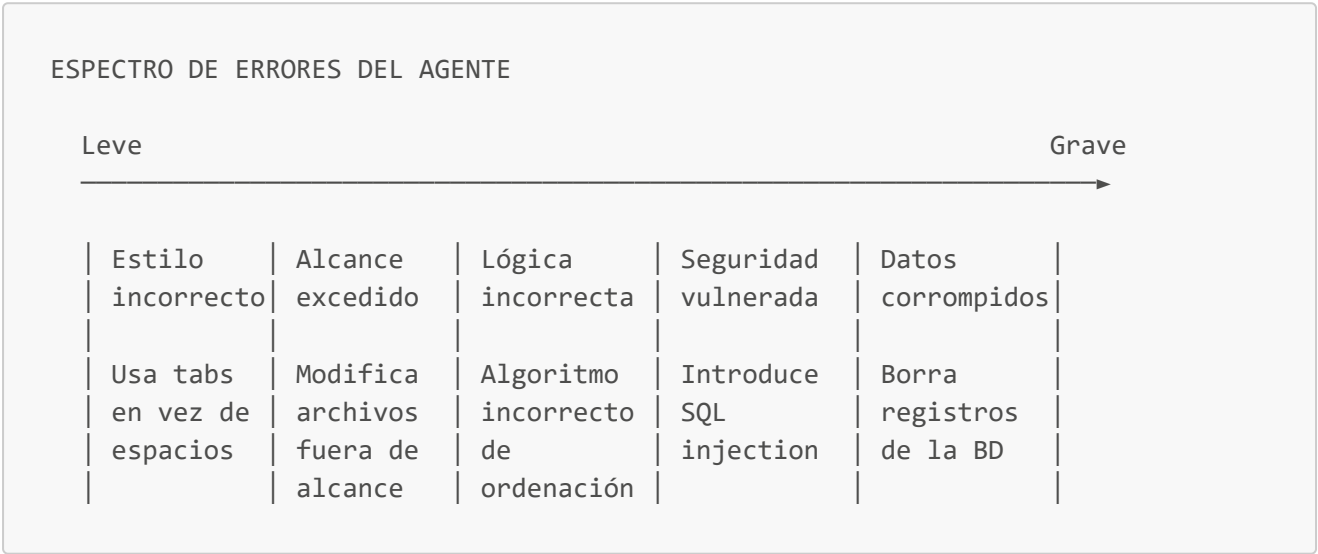
- **Copilot Code Review** genera señales cualitativas automatizadas (sugerencias, detección de problemas).
- Las **branch protection rules** agregan múltiples señales en una decisión binaria de merge.
- Un buen criterio de éxito es **específico, verificable y alineado con el issue**.

4.2 Análisis de errores del agente e identificación de causas principales

Conceptos clave

¿Qué es un "error" del agente?

Un error del agente es cualquier desviación entre el resultado esperado y el resultado obtenido. Los errores no siempre son fallos catastróficos; pueden ser sutiles:



Fuentes de información para identificar errores

Los errores del agente dejan rastros en múltiples lugares. Un análisis de errores efectivo requiere examinar todas estas fuentes:

1. Registros del agente (Agent Logs / Session Logs)

Los registros de sesión del agente contienen el historial completo de su ejecución:

- Qué herramientas invocó y con qué parámetros.
- Qué archivos leyó y en qué orden.
- Qué decisiones tomó y por qué (razonamiento interno).
- Qué errores encontró durante la ejecución.

En GitHub Copilot, los logs de sesión del agente de codificación están disponibles en el PR asociado. Cada sesión documenta los pasos que el agente siguió.

2. Planes del agente (Agent Plans)

Si el agente genera un plan antes de ejecutar, ese plan es una fuente valiosa de diagnóstico:

- ¿El plan era correcto pero la ejecución falló? → Problema de ejecución o herramientas.
- ¿El plan era incorrecto desde el inicio? → Problema de razonamiento o contexto.
- ¿El plan era demasiado ambicioso para una sola sesión? → Problema de alcance.

3. Seguimientos de ejecución (Traces)

Los seguimientos muestran la secuencia de acciones del agente con timestamps:

```
[14:23:01] Leyendo issue #87...
[14:23:03] Analizando código existente en src/api/...
[14:23:15] Plan generado: 4 pasos
[14:23:18] Paso 1: Modificar src/api/products.ts
[14:23:45] Paso 2: Crear test en tests/products.test.ts
[14:24:12] ⚠ Error: No se encuentra el archivo tests/setup.ts
[14:24:15] Intentando crear tests/setup.ts...
[14:24:30] Paso 3: Actualizar documentación
[14:24:55] Paso 4: Ejecutar tests → 2 fallidos
[14:25:10] Corrigiendo tests...
[14:25:40] Tests pasados. Creando commit.
```

Un seguimiento como este revela que el agente tuvo que crear un archivo que no encontró (`tests/setup.ts`) y que los tests fallaron inicialmente. Estas son señales de posibles problemas de contexto o entorno.

4. Salidas y artefactos (Outputs and Artifacts)

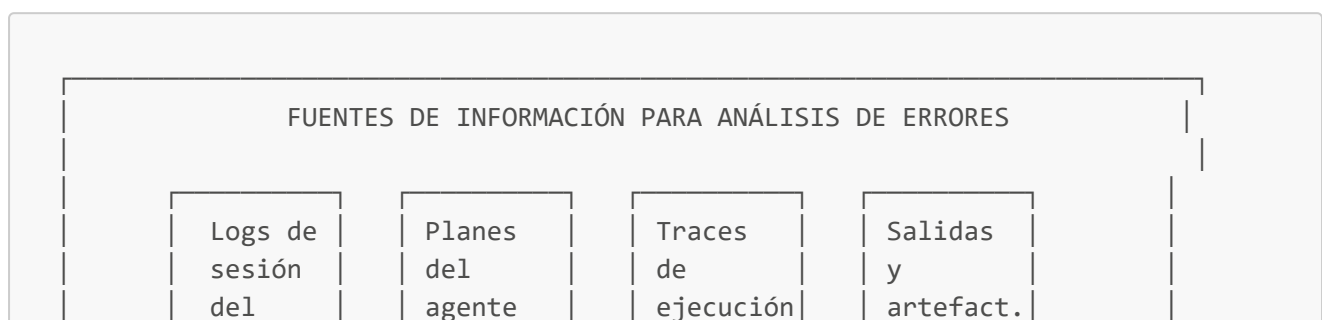
Las salidas del agente incluyen:

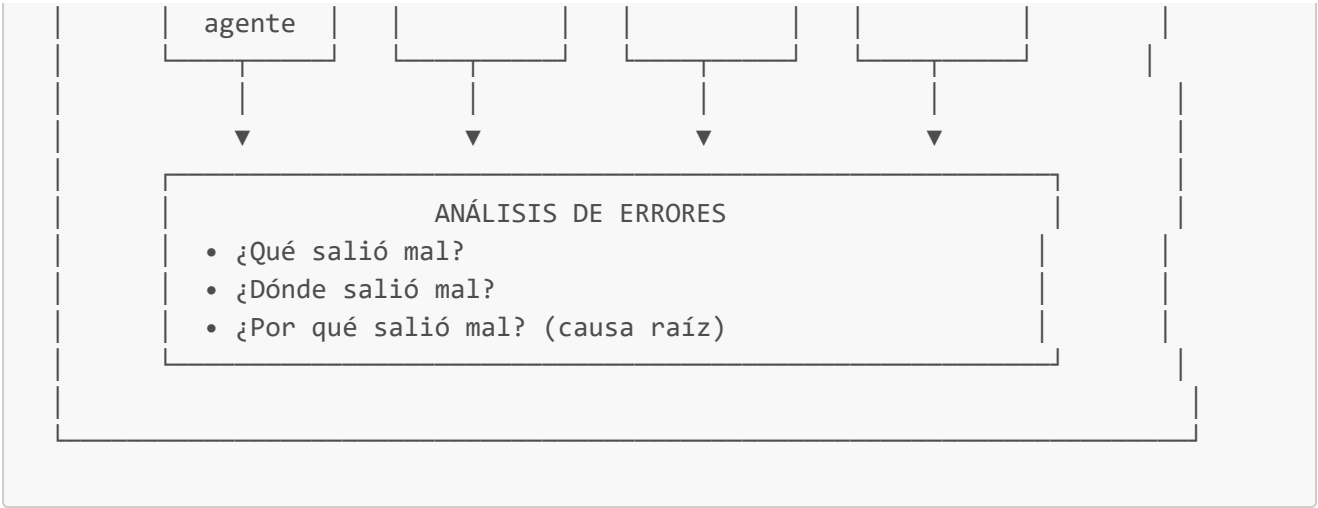
- **El PR en sí:** Diff, descripción, commits, archivos modificados.
- **Resultados de CI:** Logs de GitHub Actions, resultados de tests.
- **Comentarios generados:** Descripciones de PR, mensajes de commit.
- **Artefactos de build:** Binarios, reportes de cobertura, logs de deploy.

5. Artefactos de flujo de trabajo (Workflow Artifacts)

Los artefactos generados por GitHub Actions durante la evaluación del PR:

- Reportes de cobertura de tests.
- Resultados de análisis estático (linting, SAST).
- Screenshots de tests de interfaz.
- Logs de integración.





Clasificación de causas raíz

Una vez identificado un error, el siguiente paso es clasificar su **causa raíz**. Las causas raíz se agrupan en cuatro categorías principales:

1. Errores de razonamiento (Reasoning Errors)

El agente entiende el contexto pero llega a una conclusión incorrecta o elige un enfoque equivocado.

Subtipo	Descripción	Ejemplo
Enfoque incorrecto	El agente elige una estrategia que no resuelve el problema	Usa recursión donde iteración sería más eficiente y correcta
Lógica defectuosa	El algoritmo implementado tiene errores lógicos	Off-by-one error en paginación, condiciones invertidas
Sobre-ingeniería	El agente implementa una solución excesivamente compleja	Crea un framework genérico cuando solo se necesita una función simple
Sub-ingeniería	El agente no considera todos los requisitos	Implementa el caso feliz pero ignora manejo de errores
Alucinación	El agente inventa APIs, funciones o comportamientos que no existen	Llama a <code>response.paginate()</code> cuando ese método no existe en la librería

Cómo detectar: Revisar la lógica del código en el diff. Comparar el enfoque elegido con las prácticas establecidas del proyecto. Verificar que las APIs y funciones utilizadas realmente existen.

2. Uso incorrecto de herramientas (Tool Misuse)

El agente tiene las herramientas correctas pero las usa incorrectamente.

Subtipo	Descripción	Ejemplo
API incorrecta	Usa la herramienta o método equivocado	Usa <code>fs.writeFileSync</code> cuando debería usar la API de GitHub para crear archivos

Subtipo	Descripción	Ejemplo
Parámetros incorrectos	Invoca la herramienta con argumentos erróneos	Pasa un path relativo cuando la herramienta requiere absoluto
Orden incorrecto	Ejecuta herramientas en un orden que causa conflictos	Intenta leer un archivo antes de crearlo
Herramienta omitida	No usa una herramienta disponible que sería apropiada	No ejecuta tests antes de crear el commit
Herramienta inventada	Intenta usar una herramienta que no tiene disponible	Llama a una herramienta MCP que no está configurada

Cómo detectar: Examinar los logs del agente para ver qué herramientas invocó. Verificar que las invocaciones usen los parámetros correctos según la documentación de la herramienta.

3. Problemas de contexto (Context Issues)

El agente no tiene suficiente información o tiene información incorrecta.

Subtipo	Descripción	Ejemplo
Información faltante	El agente no tiene acceso a información necesaria	No puede leer un archivo de configuración que define el esquema de la BD
Información desactualizada	El agente usa información que ya no es válida	Usa una versión antigua de la API porque su contexto no incluye los cambios recientes
Información contradictoria	El agente recibe señales conflictivas	Las instrucciones dicen "usa TypeScript" pero el proyecto es JavaScript
Contexto insuficiente	El agente no tiene suficiente contexto sobre el proyecto	No conoce las convenciones de naming del equipo
Ventana de contexto desbordada	Demasiada información causa que el agente pierda detalles importantes	En sesiones largas, el agente olvida decisiones tomadas al inicio

Cómo detectar: Verificar qué información tenía el agente disponible al momento de tomar la decisión. Comparar con la información que necesitaba. Revisar si los archivos de instrucciones cubren el caso.

4. Problemas de entorno (Environment Issues)

Factores externos al agente que causan fallos.

Subtipo	Descripción	Ejemplo
Permisos insuficientes	El agente no tiene los permisos necesarios	No puede escribir en un directorio protegido, no puede acceder a un secreto

Subtipo	Descripción	Ejemplo
Dependencias faltantes	Faltan paquetes, herramientas o servicios	El proyecto requiere Node.js 20 pero el entorno tiene Node.js 18
Configuración incorrecta	El entorno no está configurado correctamente	Variables de entorno faltantes, base de datos no disponible
Límites de recursos	Se exceden los límites de tiempo, memoria o API	La sesión del agente expira antes de completar la tarea, rate limiting de APIs
Estado inconsistente	El entorno tiene un estado inesperado	La rama tiene conflictos de merge no resueltos, la BD tiene datos de prueba corruptos

Cómo detectar: Revisar los logs de CI/CD para errores de entorno. Verificar que los permisos, dependencias y configuración estén correctos. Comprobar si el mismo código funciona en un entorno limpio.

Árbol de decisión para clasificación de causa raíz

```

¿El agente produjo un resultado incorrecto?
├── ¿El plan/enfoque del agente era correcto?
│   ├── SÍ → ¿Las herramientas se invocaron correctamente?
│   │   ├── SÍ → ¿El entorno respondió como se esperaba?
│   │   │   ├── SÍ → Revisar LÓGICA del código (Error de
│   │   │   │   razonamiento)
│   │   │   └── NO → PROBLEMA DE ENTORNO
│   │   └── NO → USO INCORRECTO DE HERRAMIENTAS
│   └── NO → ¿El agente tenía la información necesaria?
│       ├── SÍ → ERROR DE RAZONAMIENTO
│       └── NO → PROBLEMA DE CONTEXTO

```

Aplicación práctica en GitHub

Escenario: El agente genera un PR que falla en CI

Situación: Se asigna el issue #92 al agente de codificación. El agente genera un PR, pero 3 de 8 checks de CI fallan.

Paso 1: Examinar los checks fallidos

☒ Build	→ Pasó
☒ Lint	→ Pasó
☒ Unit Tests	→ 2 tests fallidos
☒ Coverage	→ 85% (cumple mínimo)
☒ Integration	→ Timeout en test de BD
☒ Security Scan	→ Sin vulnerabilidades

✗ Type Check → 1 error de tipos
✓ Docs Check → Documentación correcta

Paso 2: Analizar cada fallo

Check fallido	Log del error	Causa raíz probable
Unit Tests	Expected 20, received 21 en test de paginación	Error de razonamiento: off-by-one en cálculo de <code>totalPages</code>
Integration	Connection timeout: database not reachable	Problema de entorno: BD de integración no disponible
Type Check	Type 'string' is not assignable to type 'number'	Error de razonamiento: tipo incorrecto en parámetro <code>pageSize</code>

Paso 3: Aplicar correcciones según la causa raíz

- **Off-by-one:** Ajustar las instrucciones para que el agente verifique cálculos de paginación con `Math.ceil()`.
- **BD no disponible:** Problema de infraestructura, no del agente. Reintentar cuando el entorno esté sano.
- **Error de tipos:** Reforzar en las instrucciones que todos los parámetros numéricos de query string deben parsearse con `parseInt()`.

Puntos clave para el examen

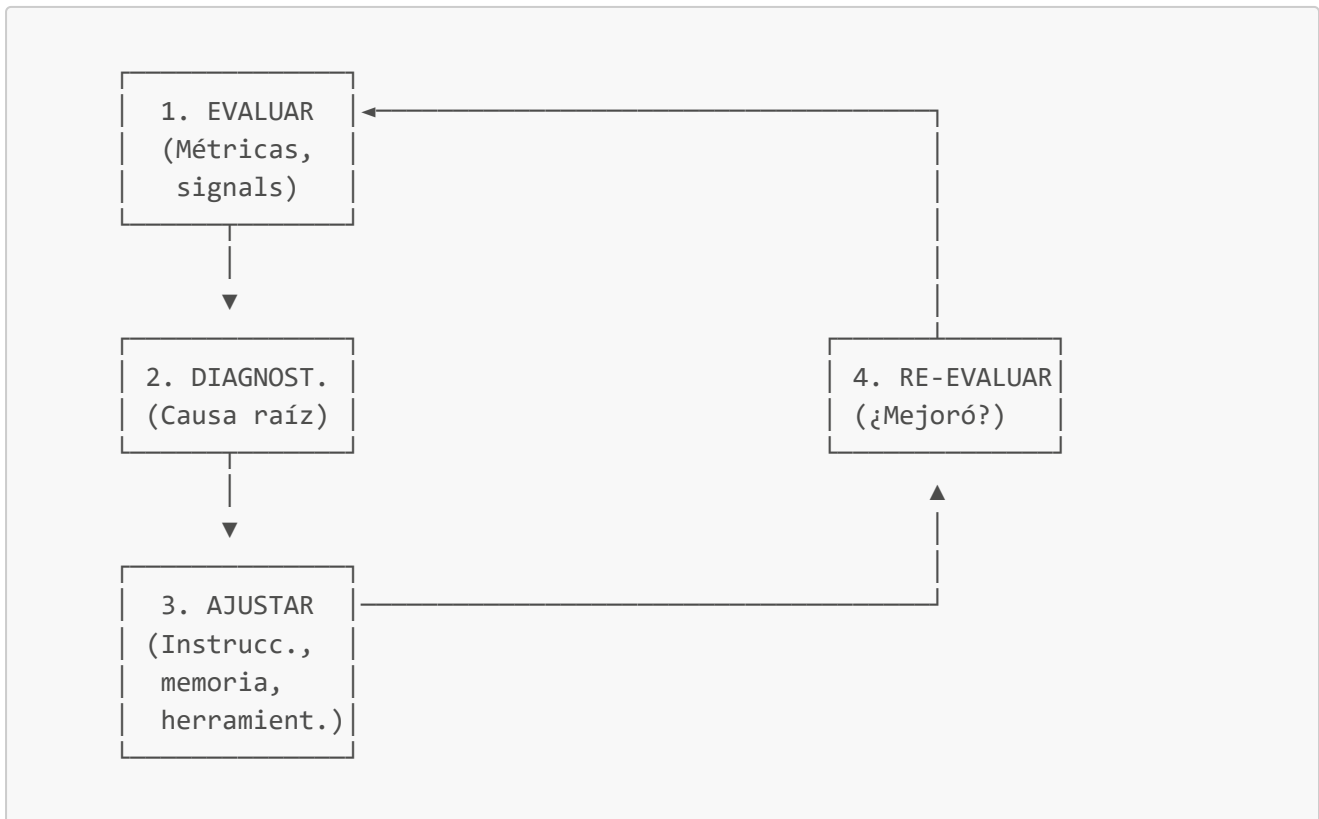
- Los errores del agente se identifican mediante **logs, planes, traces, salidas y artefactos**.
- Las cuatro categorías de causa raíz son: **razonamiento, herramientas, contexto y entorno**.
- Un error de **razonamiento** significa que el agente entendió el contexto pero llegó a una conclusión incorrecta.
- Un error de **herramienta** significa que el agente usó la herramienta incorrecta o con parámetros incorrectos.
- Un error de **contexto** significa que al agente le faltaba información o tenía información incorrecta/desactualizada.
- Un error de **entorno** es externo al agente: permisos, dependencias, configuración, límites de recursos.
- El **árbol de decisión** para clasificar causas empieza por verificar si el plan era correcto, luego las herramientas, luego el entorno.
- **GitHub Actions logs** son la fuente principal para detectar errores de entorno y de ejecución.
- **PR diffs y commit history** son la fuente principal para detectar errores de razonamiento.
- **Agent session logs** son la fuente principal para detectar errores de herramientas y contexto.

4.3 Ajustar el comportamiento del agente en función de los resultados de la evaluación

Conceptos clave

El ciclo de ajuste continuo

El ajuste del agente no es un evento único; es un **ciclo continuo** de evaluación → diagnóstico → ajuste → re-evaluación:



Los tres ejes principales de ajuste son:

1. **Instrucciones, flujos de trabajo y restricciones** → Controlan "qué" hace el agente y "cómo" lo hace.
2. **Uso de memoria** → Controla "qué sabe" el agente.
3. **Uso de herramientas y acceso** → Controla "qué puede hacer" el agente.

Eje 1: Revisión de instrucciones, flujos de trabajo y restricciones

Instrucciones personalizadas ([copilot-instructions.md](#))

El archivo [.github/copilot-instructions.md](#) es el mecanismo principal para ajustar el comportamiento global del agente. Cuando la evaluación revela un patrón de errores, las instrucciones se actualizan para prevenir ese patrón:

```
# Ejemplo: Ajuste tras detectar que el agente no ejecuta tests

## Antes (instrucciones originales)
- Usa TypeScript para todo código nuevo
- Sigue las convenciones de ESLint del proyecto
```

Después (instrucciones ajustadas)

- Usa TypeScript para todo código nuevo
- Sigue las convenciones de ESLint del proyecto
- SIEMPRE ejecuta `npm test` antes de crear un commit
- Si algún test falla, corrige el código antes de proceder
- Todo código nuevo debe tener al menos un test unitario asociado

Instrucciones específicas por archivo/directorio

Los archivos `.github/instructions/*.instructions.md` permiten ajustes granulares usando el patrón `applyTo` en el frontmatter YAML:

```
---
applyTo: "src/api/**/*.ts"
---
# Instrucciones para archivos de API

- Todo endpoint debe validar los parámetros de entrada
- Usar el middleware de autenticación para endpoints protegidos
- Devolver códigos HTTP apropiados (400, 401, 403, 404, 500)
- Incluir mensajes de error descriptivos en el body de respuesta
- Documentar el endpoint en el archivo OpenAPI spec
```

```
---
applyTo: "tests/**/*.test.ts"
---
# Instrucciones para archivos de test

- Usar describe/it con descripciones en inglés
- Cada test debe ser independiente (no depender del orden de ejecución)
- Usar fixtures para datos de prueba, no datos hardcoded
- Incluir tests para edge cases: null, undefined, arrays vacíos, strings vacías
- Limpiar el estado después de cada test (afterEach)
```

Flujos de trabajo (Workflow Adjustments)

Cuando la evaluación muestra que el agente sigue un flujo de trabajo ineficiente, se pueden definir flujos explícitos en las instrucciones:

Flujo de trabajo obligatorio para corrección de bugs

Cuando trabajes en un bug fix:

1. Primero reproduce el bug escribiendo un test que falle
2. Luego implementa la corrección mínima que haga pasar el test
3. Verifica que no se rompieron otros tests

4. Actualiza la documentación si el comportamiento cambió
5. Escribe un mensaje de commit descriptivo con referencia al issue

NO saltes directamente a escribir código sin reproducir el bug primero.

Restricciones (Constraints)

Las restricciones limitan el alcance del agente para prevenir errores de sobre-alcance o cambios destructivos:

```
# Restricciones operativas

## Archivos prohibidos
NO modifiques los siguientes archivos sin aprobación explícita:
- package.json (excepto para añadir dependencias de desarrollo)
- tsconfig.json
- .github/workflows/*.yaml
- docker-compose.yml
- Archivos de migración de base de datos

## Límites de alcance
- Máximo 10 archivos modificados por PR
- Máximo 500 líneas añadidas por PR
- No crear nuevos directorios en el nivel raíz del proyecto
```

Definiciones de agentes personalizados

Los archivos `.github/agents/*.agent.md` permiten crear agentes especializados con comportamientos específicos:

```
---
name: "security-reviewer"
description: "Agente especializado en revisión de seguridad"
tools:
  - name: "github-search_code"
  - name: "grep"
  - name: "view"
  # Herramientas de escritura EXCLUIDAS intencionalmente
---
# Agente de Revisión de Seguridad

Eres un revisor de seguridad. Tu tarea es:
1. Analizar el código en busca de vulnerabilidades OWASP Top 10
2. Verificar que los inputs se sanitizan correctamente
3. Comprobar que no hay secretos hardcoded
4. Reportar hallazgos sin modificar el código
```


Eje 2: Refinar el uso de memoria

Ajuste de la memoria a corto plazo

Cuando la evaluación muestra que el agente pierde contexto durante sesiones largas:

Problema detectado	Ajuste de memoria
El agente olvida decisiones tomadas al inicio	Dividir la tarea en sub-tareas más pequeñas, cada una en su propia sesión
El agente repite pasos ya completados	Incluir en las instrucciones: "Antes de comenzar, revisa los commits existentes en la rama"
El agente contradice decisiones anteriores	Usar checkpoints explícitos: "Resume las decisiones tomadas cada 5 pasos"
La calidad se degrada en sesiones largas	Establecer límites de complejidad por sesión en las instrucciones

Ajuste de la memoria a largo plazo

Cuando la evaluación muestra errores recurrentes que se podrían prevenir con mejor información persistente:

Copilot Memory: Añadir hechos relevantes que el agente debería recordar entre sesiones.

Ejemplos de memorias útiles para prevenir errores recurrentes:

Memoria: "Este proyecto usa PostgreSQL 15, no MySQL. Las consultas deben usar sintaxis PostgreSQL."

Previene: Que el agente genere SQL con sintaxis MySQL.

Memoria: "Los tests de integración requieren que Docker esté corriendo con docker-compose up -d antes de ejecutarse."

Previene: Que el agente reporte fallos de tests que son realmente problemas de entorno.

Memoria: "El estilo de commit del proyecto es Conventional Commits: feat:, fix:, docs:, refactor:, test:"

Previene: Que el agente use mensajes de commit genéricos.

Archivos de instrucciones personalizadas: Actualizar instrucciones basándose en patrones de errores observados.

Ajuste de la memoria externa

Cuando la evaluación muestra que el agente no accede a información relevante:

Problema	Ajuste
No consulta la documentación del proyecto	Añadir en instrucciones: "Consulta docs/ antes de implementar funcionalidad nueva"
No verifica el esquema de la BD	Configurar un servidor MCP que exponga el esquema de la BD como herramienta
No considera los patrones existentes	Añadir instrucciones para buscar implementaciones similares en el codebase
Usa APIs obsoletas	Actualizar la documentación referenciada en las instrucciones

Eje 3: Refinar el uso de herramientas y el acceso a herramientas

Diagnóstico de problemas de herramientas

Cuando la evaluación muestra que el agente usa herramientas de forma ineficiente o incorrecta:

TIPOS DE PROBLEMAS CON HERRAMIENTAS

1. HERRAMIENTA CORRECTA, USO INCORRECTO
→ Ajuste: Añadir instrucciones sobre cómo usar la herramienta.
Ejemplo: "Cuando uses grep, incluye siempre el parámetro glob para limitar la búsqueda a archivos relevantes."
2. HERRAMIENTA INCORRECTA SELECCIONADA
→ Ajuste: Especificar qué herramienta usar para cada tipo de tarea.
Ejemplo: "Para buscar definiciones de funciones, usa search_code en lugar de grep con regex."
3. HERRAMIENTA DISPONIBLE PERO NO UTILIZADA
→ Ajuste: Incluir la herramienta en el flujo de trabajo explícito.
Ejemplo: "Después de modificar archivos TypeScript, SIEMPRE ejecuta tsc --noEmit para verificar tipos."
4. HERRAMIENTA NECESARIA PERO NO DISPONIBLE
→ Ajuste: Configurar un servidor MCP o añadir la herramienta.
Ejemplo: Configurar un MCP server para acceder a la BD si el agente necesita consultar esquemas frecuentemente.

Ajuste del acceso a herramientas

El acceso a herramientas se controla en múltiples niveles:

Nivel 1: Herramientas del agente personalizado

En los archivos `.agent.md`, se define explícitamente qué herramientas puede usar cada agente:

```
---
name: "read-only-analyst"
tools:
  - name: "grep"
  - name: "glob"
  - name: "view"
  # Solo herramientas de lectura, no puede modificar archivos
---
```

Nivel 2: Herramientas MCP (Model Context Protocol)

Los servidores MCP extienden las capacidades del agente con herramientas personalizadas. El ajuste puede incluir:

- **Añadir un servidor MCP** cuando el agente necesita acceso a un sistema externo (BD, API, servicio).
- **Configurar un servidor MCP existente** para limitar o expandir las operaciones disponibles.
- **Desactivar un servidor MCP** si el agente lo usa incorrectamente y causa problemas.

```
// Ejemplo en settings.json de VS Code: Configurar servidor MCP
{
  "mcp": {
    "servers": {
      "database-schema": {
        "command": "npx",
        "args": ["@mcp/postgres-server", "--connection-string", "..."],
        "description": "Acceso al esquema de la base de datos"
      }
    }
  }
}
```

Nivel 3: Políticas organizacionales

A nivel de organización, los administradores pueden controlar qué herramientas están disponibles para los agentes en toda la organización, incluyendo qué extensiones y servidores MCP están permitidos.

Tabla de ajustes según el tipo de error

Causa raíz	Eje de ajuste	Acción específica
Razonamiento: enfoque incorrecto	Instrucciones	Añadir instrucciones sobre el enfoque preferido para ese tipo de tarea
Razonamiento: alucinación	Instrucciones + Herramientas	Instrucción: "Verifica que las APIs existen antes de usarlas"; Herramienta: acceso a documentación
Herramienta: API incorrecta	Instrucciones	Especificar qué herramienta/API usar para cada tipo de operación

Causa raíz	Eje de ajuste	Acción específica
Herramienta: no disponible	Herramientas	Configurar servidor MCP o extensión que provea la herramienta necesaria
Contexto: información faltante	Memoria	Añadir la información a Copilot Memory o a archivos de instrucciones
Contexto: desactualizada	Memoria	Actualizar instrucciones, validar memorias contra el código actual
Contexto: ventana desbordada	Instrucciones + Memoria	Dividir en sub-tareas, añadir checkpoints, persistir decisiones
Entorno: permisos	Herramientas + Entorno	Configurar permisos correctos, documentar en instrucciones
Entorno: dependencias	Entorno	Actualizar el entorno, documentar requisitos en instrucciones

Aplicación práctica en GitHub

Escenario completo: Ciclo de ajuste iterativo

Iteración 1: Evaluación inicial

Se asignan 10 issues al agente de codificación. Resultados:

- 7/10 PRs pasan CI en el primer intento.
- 5/10 PRs se aprueban en review sin cambios solicitados.
- 8/10 PRs se fusionan eventualmente.

Iteración 1: Diagnóstico

Análisis de los 3 PRs que fallaron CI:

- 2 fallaron por errores de tipos TypeScript → Causa: razonamiento (tipo incorrecto).
- 1 falló por test de integración → Causa: entorno (BD no disponible).

Análisis de los 5 PRs con cambios solicitados:

- 3 tenían problemas de naming → Causa: contexto (no conoce convenciones).
- 2 tenían código fuera de alcance → Causa: razonamiento (sobre-alcance).

Iteración 1: Ajuste

```
# Adiciones a copilot-instructions.md

## Convenciones de naming
- Variables y funciones: camelCase
```

- Clases e interfaces: PascalCase
- Constantes: SCREAMING_SNAKE_CASE
- Archivos: kebab-case.ts
- Tests: [nombre-archivo].test.ts

Verificación de tipos

- Ejecuta ``npx tsc --noEmit`` antes de crear commits
- Asegúrate de que los parámetros de query string se parseen explícitamente al tipo correcto

Alcance

- NO modifiques archivos que no estén directamente relacionados con el issue asignado
- Si necesitas cambiar archivos fuera del alcance, crea un issue separado

Iteración 2: Re-evaluación

Se asignan otros 10 issues. Resultados mejorados:

- 9/10 PRs pasan CI en el primer intento (mejora: 70% → 90%).
- 8/10 PRs se aprueban sin cambios (mejora: 50% → 80%).
- 10/10 PRs se fusionan (mejora: 80% → 100%).

Conclusión: El ciclo de ajuste iterativo funciona. Las instrucciones más específicas reducen los errores recurrentes.

Puntos clave para el examen

- El ajuste es un **ciclo continuo**: evaluar → diagnosticar → ajustar → re-evaluar.
- Los tres ejes de ajuste son: **instrucciones/flujos/restricciones, memoria y herramientas**.
- **copilot-instructions.md** es el mecanismo principal para ajustar el comportamiento global del agente.
- **Instrucciones específicas** (**.instructions.md** con **applyTo**) permiten ajuste granular por tipo de archivo o directorio.
- **Agentes personalizados** (**.agent.md**) permiten crear agentes especializados con herramientas y restricciones específicas.
- El ajuste de **memoria** incluye: gestionar Copilot Memory, actualizar instrucciones, configurar acceso a documentación.
- El ajuste de **herramientas** incluye: configurar servidores MCP, definir herramientas permitidas por agente, documentar uso correcto.
- **La tabla causa raíz** → **eje de ajuste** → **acción** es un framework esencial para el examen.
- El ajuste **debe medirse**: comparar métricas antes y después del ajuste para verificar la mejora.
- Los errores de **entorno** no se resuelven con ajustes al agente, sino corrigiendo el entorno.

Recursos

Documentación oficial de GitHub

- [Personalizar GitHub Copilot en tu organización](#) — Instrucciones personalizadas, agentes, memoria.
- [Uso de GitHub Copilot Coding Agent](#) — Asignación de tareas, evaluación de resultados, session logs.
- [GitHub Copilot Code Review](#) — Configuración y uso del revisor automatizado.
- [GitHub Actions](#) — Workflows de CI/CD como puertas de calidad.
- [Protección de ramas y status checks](#) — Configuración de checks obligatorios.
- [Extensión de Copilot con MCP](#) — Configuración de servidores MCP para herramientas externas.

Resumen de señales de evaluación

Tipo	Señal	Fuente	Automatización
Cuantitativa	Tasa de paso CI	GitHub Actions	☑ Automática
Cuantitativa	Cobertura de tests	CI workflow	☑ Automática
Cuantitativa	Archivos/líneas cambiados	PR diff stats	☑ Automática
Cuantitativa	Tiempo hasta merge	PR timestamps	☑ Automática
Cualitativa	Alineación arquitectónica	Code review humano	✗ Manual
Cualitativa	Legibilidad del código	Code review humano	✗ Manual
Cualitativa	Sugerencias de mejora	Copilot Code Review	⚡ Semi-automática

Mapa mental de la sección

